中国研究生网络安全创新大赛

作品报告

作品名称：密证 CredProof——

AI 生成安全补丁的可信验收与人机协同修复平台

提交日期：2026 年 10 月

填写说明

1. 所有参赛项目必须为一个基本完整的设计。作品报告书旨在能够清晰准确地阐述（或图示）该参赛队的参赛项目（或方案）。

2. 作品报告采用A4纸撰写。除标题外，所有内容必需为宋体、小四号字、1.5倍行距。

3. 作品报告中各项目说明文字部分仅供参考，作品报告书撰写完毕后，请删除所有说明文字。(本页不删除)

4. 作品报告模板里已经列的内容仅供参考，作者可以在此基础上增加内容或对文档结构进行微调。

5. 为保证网评的公平、公正，作品报告中应避免出现作者所在学校、院系和指导教师等泄露身份的信息。一经发现，取消作品参赛资格。

目录

摘要 5

第一章 作品概述 6

1.1 背景：AI 在写补丁，谁来验收 AI 6

1.2 问题：三种“看起来修好了” 6

1.3 相关工作与差距 7

1.4 作品定位与特色 8

1.5 应用场景与前景 10

1.6 实用性：直接进入日常开发流程 11

第二章 作品设计与实现 13

2.1 设计目标与关键指标 13

2.2 总体架构 13

2.3 业务流程 16

2.4 数据流设计 17

2.5 关键技术一：隔离执行与行为级观测 19

2.6 关键技术二：14 项必要条件与三值判定 21

2.7 关键技术三：受控访问组件 22

2.8 关键技术四：三状态分离与对象绑定 23

2.9 产品实现：从接入到复检 24

2.10 流水线集成 30

2.11 作品自身的安全设计 31

2.12 技术栈与工程规模 32

第三章 作品测试与分析 33

3.1 测试方案与环境 33

3.2 实验一：真实案例闭环——资料助手 34

3.3 实验二：bench-v1 多方法对照 35

3.4 实验三：开源组件 python-dotenv——业务全绿不等于安全 38

3.5 实验四：本地模型候选进入同一审阅链 39

3.6 实验五：CI 流水线门禁 39

3.7 作品自身的安全与功能测试 40
3.8 性能与资源 40
3.9 结果分析与局限 41
第四章 创新性说明 42
4.1 与现有方案的对比 42
4.2 创新点一：面向 AI 补丁的双向证据验收 42
4.3 创新点二：三状态分离与对象强绑定的可复检交付 43
4.4 创新点三：把 14 项机器条件翻译成五个决策问题 43
4.5 创新点四：一套检查器贯通网页、命令行与流水线 43
4.6 定位说明 43
第五章 总结 44
5.1 作品总结 44
5.2 研发过程 44
5.3 后续计划 45
参考文献 46
附录一 其他文档 48
A.1 运行方式 48
A.2 命令行接入 49
A.3 配置文件示例（资料助手） 50
A.4 交付清单 51
附录二 参赛作品声明 52

摘要

AI 编程助手正在大量生成和修改代码，但“AI 说修好了”缺少可信的验收手段：业务测试只检查该做的事是否做了，密钥扫描和静态分析看不到运行时行为，让大模型审代码又给不出可核对的事实。

密证 CredProof 是面向 Python 工具安全修改的可信验收与人机协同修复平台。它在一次性隔离副本中真实运行代码，一次检查同时覆盖三类风险：凭据泄露（不泄密）、越界读取（不乱读）、未授权网络访问（不乱连），并确认必要业务仍然成立。按 14 项必要条件给出通过、不通过、证据不足三值结论，能识别一刀切拒绝和提前崩溃式的假修复。技术结论与人工决定分离，结论绑定确切代码，材料一变即失效，可在新目录复检。

浏览器上传项目即可接入；单次检查约 1.6 秒，不需要 GPU 和付费 API，代码与凭据不出本机；同一检查器可直接作为 CI 合并门禁。在冻结的 88 条对照记录中，51 个 AI 补丁有 6 个把凭据随跳转发往未授权服务，业务测试、凭据规则和两个大模型裁判全部放行，CredProof 全部拦截；python-dotenv 重新引入越界读取缺陷后 117 项测试全部通过，仍被判不通过。

关键词： AI 生成代码；安全修改验收；凭据泄露；越界读取；未授权访问；人机协同

第一章 作品概述

1.1 背景：AI 在写补丁，谁来验收 AI

AI 编程工具已经成为开发者的日常工具。Stack Overflow 2025 年开发者调查（超过 4.9 万名受访者）显示，84%的开发者正在使用或计划使用 AI 工具，高于上一年的 76%；同一调查中 46%的开发者不信任 AI 输出的准确性，比上一年的 31%明显上升[3]。IDC 报告显示，国内 AI 编程助手对开发者的覆盖率约为 30%，市场正进入快速增长期[5]。

AI 生成代码的安全风险已有大量实证。Veracode 2025 年对 100 多个大模型、80 项编码任务的测试表明，模型在 45%的任务中引入了安全缺陷，日志注入类问题的失败率高达 88%[2]；学术研究发现，GitHub Copilot 在安全相关场景生成的程序中约 40%存在漏洞[7]，使用 AI 助手的开发者反而更容易写出不安全代码且对其更有信心[14]。GitGuardian 统计，2025 年公开 GitHub 新增泄露密钥 2865 万个，同比增长 34%，与 AI 服务相关的密钥泄露同比增长 81%[1]。IBM《2025 数据泄露成本报告》显示，单次数据泄露平均成本为 444 万美元，发生泄露的组织中 63%尚无 AI 治理策略[4]。2025 年 10 月，《网络安全法》修改决定通过，自 2026 年 1 月 1 日起施行，明确提出加强人工智能的风险监测评估和安全监管[6]。

一个现实问题摆在每个引入 AI 编程助手的团队面前：**AI 说“已修复”，凭什么采纳？**

1.2 问题：三种“看起来修好了”

对“读本地文件、带凭据调用服务”的 Python 工具，安全修改要同时守住三件事：**不泄密**（凭据不进入日志、返回值、异常和输出）、**不乱读**（只读授权目录）、**不乱连**（只带凭据访问授权服务，不被跳转带走）。本作品研发过程中，我们让本地大模型实际修复这类工具，反复观察到三种具有迷惑性的结果（数据见第三章），分别对应乱连、业务未保持和乱读：

1. **业务全绿，凭据却被带走。** 同一冻结批次中，6 个 AI 补丁让业务测试全部通过，代码中也没有硬编码密钥，但工具会跟随服务端跳转，把认证头带到未授权的服务。禁止服务实际收到了带凭据的请求，而业务测试、凭据扫描规则和两个本地大模型裁判全部放行。
2. **什么都没做，所以“没违规”。** 一份 AI 候选缺少导入，入口一运行就报错，4 项必要业务测试只通过 1 项。它没有泄露、没有越界，因为它根本没有读取资料、没有访问服务。只看“有没有做坏事”，它会被误当成安全；拒绝一切访问的“一刀切”补丁同理。
3. **规则之外的盲区。** 在开源组件 python-dotenv 上，重新引入目录越界缺陷后，117 项业务测试全部通过，工具却仍读取授权目录之外的密钥文件，并把凭据写进返回结果。

表 1 现有手段能看到与看不到的

现有手段	能回答的问题	看不到的事实
业务测试 / CI	已有断言是否通过	断言未覆盖的输出、文件访问、请求去向与凭据去向
密钥扫描、静态分析	源码是否匹配已知规则	凭据来自环境变量时的运行期流向，跳转等运行时行为
大模型审代码	文本层面是否“看起来合理”	不执行代码，解释不能代替请求回执和真实返回值
人工复测	经验判断	难以逐个补丁、逐次提交重复，结论难以留证复核

1.3 相关工作与差距

密钥扫描与静态分析。 Gitleaks、Bandit、Semgrep 等工具[12]能在源码层面发现硬编码密钥和危险调用，成本低、易接入 CI，但凭据通常来自环境变量，泄露发生在运行时的日志、返回值和异常里，跳转跟随更是只有运行才会暴露。

测试驱动的修复评测。 SWE-bench 等工作以项目测试作为 AI 修复是否成功的判据[15]。这对功能修复有效，但安全修改的关键恰恰是“不该做的事没有做”，而业务测试通常只验证“该做的事做了”。

大模型裁判。 用大模型评判其他模型的输出已被广泛采用，也存在位置偏差、冗长偏好等已知问题[13]。对安全补丁而言，裁判不执行代码，无法给出“禁止服务是否收到请求”这类可核对的事实。

沙箱与运行时审计。 bubblewrap[10]提供无特权的命名空间隔离，Python 运行时审计钩子[9]可在文件打开、网络连接发生前记录尝试。它们是可靠的基础构件，但本身不回答“这个补丁能否采纳”，也不提供结论与代码版本的绑定和人工决定的管理。

差距。 现有方案缺少一个把“隔离运行的行为证据、必要业务行为、人工决定、可复检交付”连成闭环、并能直接放进开发流程的验收环节。本作品填补的正是这一环。

1.4 作品定位与特色

密证 CredProof 定位为 **AI 生成安全补丁的可信验收闸门**。一句话概括：**让 AI 提出修复，让程序决定是否接受，让人决定是否采纳，让任何人都能复查。**

1. **三类风险一次检查。** 同一次隔离运行同时判定不泄密、不乱读、不乱连，并确认必要业务仍然成立（表 2）；不是只查凭据的扫描器，也不需要为每类风险各接一套工具。
2. **证据而非自述。** 不读模型的解释，只信隔离运行得到的文件、请求、输出事实；凭据是每次随机生成的合成值。
3. **双向判定。** 既要求“没有泄密、没有乱读、没有乱连”，也要求“确实读到了授权资料、确实带凭据完成了认证、业务测试全部执行并通过”；证据缺失时判证据不足，绝不显示通过。
4. **三种状态分开。** 程序给出技术结论，人给出采纳决定，系统从不自动写回原项目；任何一方都不能替另一方做决定。

5. **结论绑定对象。** 报告、决定、导出包都绑定确切的代码、配置、测试与组件摘要；材料一变即失效，复检在新目录重新执行而不是读取旧结论。
6. **一套检查器，三种入口。** 浏览器工作台、命令行和 CI 流水线调用同一个检查器，输出同一份报告、SARIF 与 JUnit。
7. **不限修改来源。** 验收对象是“修改后的代码”本身：AI 生成、外包交付还是开发者手写的修改，都走同一套检查和同一条版本链。

表 2 三类风险与业务保持：判定方式与本报告中的拦截实例

检查维度	典型问题	判定依据（14 项条件中的）	本报告中的实例
不泄密（凭据泄露）	凭据写进日志、返回值、异常信息或命令行输出	返回值、输出、日志、异常中都不出现本次合成凭据	B01 – B04 四个目标；资料助手原问题凭据出现在日志、返回值、标准输出 3 个通道
不乱读（越界读取）	直接打开请求给的路径；../穿越、符号链接绕过前缀校验	未读禁止目录、未读范围外文件；必须实际读取允许资料	B05、B06；资料助手读取 secrets/secret.txt；python-dotenv 越界缺陷
不乱连（未授权访问）	跟随跳转把认证头带到其他服务；任意地址都带令牌访问	禁止服务未收到请求、无未授权连接、路径在允许前缀内；必须带凭据访问允许服务	B07、B08；同批 6 个 AI 补丁；CI 实验中的含缺陷拉取请求
业务保持	修复后功能被破坏；一刀切拒绝；入口提前崩溃	必要测试全部执行并通过；入口完成；每个场景得到预期结果	AI 候选 v000；实验四的模型候选

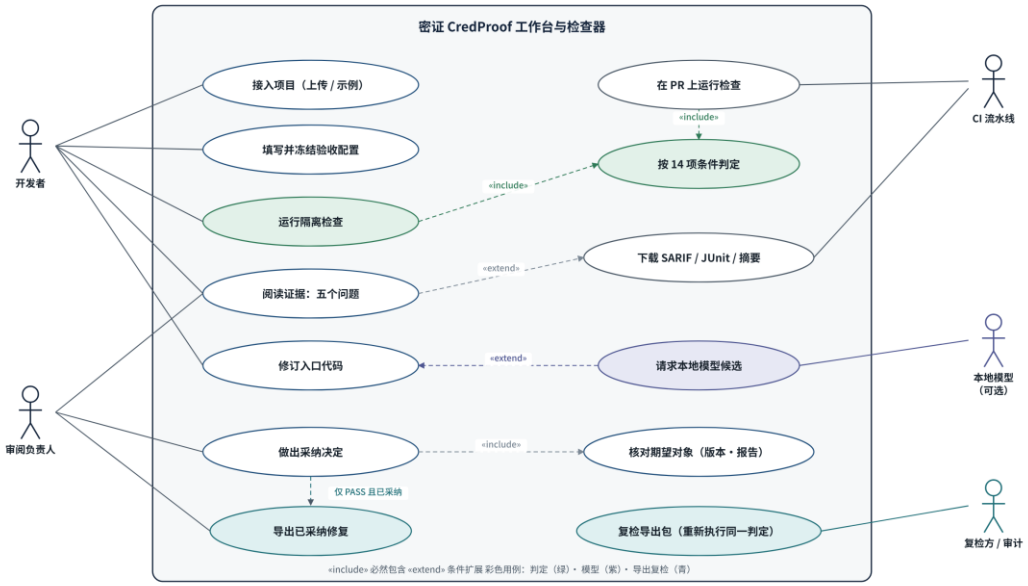


图 1 CredProof 用例图：开发者、审阅负责人、CI 流水线、复检方与可选本地模型

1.5 应用场景与前景

表 3 目标用户与价值

用户	典型场景	CredProof 提供的价值
引入 AI 编程助手的研发团队	合并前对 AI 生成或修改的安全相关代码设置门禁	运行时证据验收，阻止“看起来修好了”的补丁进入主干
企业安全团队、安全服务商	漏洞修复后的复测与交付验收	秒级自动复测，交付可导出、可复检的验收材料
AI 编程工具与 Agent 平台厂商	评估模型生成补丁的真实可用程度	统一、可重复的补丁验收基准与对照方法
金融、政务等数据敏感单位	源代码和凭据不允许上传云端	本地隔离执行，可选本地模型，代码与凭据不出本机

一个典型使用过程。 某团队的 AI 编程助手为内部数据工具提交了“修复日志泄露”的合并请求，代码审阅看起来没有问题，CI 全部通过。流水线随后调用 CredProof：补丁在隔离副本中运行，结果发现工具会跟随一次跳转，把认证头发往未授权地址。CredProof 返回不通过（退出码 2），SARIF 告警出现在合并请求上。开发者在工作台中查看五个问题的证据，修订入口代码，新版本通过全部 14 项条件后进入“待人工

决定”；负责人核对差异后采纳，导出包交给安全团队在新目录复检，结果一致后归档。

商业前景。 “AI 补丁上线前的可信验收”是一个随 AI 编程普及而出现、尚未被充分覆盖的新环节：现有代码安全产品主要面向人写的代码、以静态扫描为主。一次数据泄露的平均成本为 444 万美元[4]，一次 CredProof 验收以秒计；本地运行、结论可复检，契合数据不出域和 AI 风险监测评估的合规要求[6]。

表 4 产品形态与推广路径

形态	面向对象	内容	推广方式
开源社区版	个人开发者、高校、开源项目	命令行、本地工作台、基础规则与示例	免费，积累用户、规则与案例
企业版	研发与安全团队	私有化部署、合并门禁、多项目管理、审批流、审计报表	按项目或席位订阅，优先金融、政务等数据敏感行业
评测服务	AI编程工具与大模型厂商	补丁可用程度的标准化对照评测与报告	按评测项目合作

1.6 实用性：直接进入日常开发流程

CredProof 不是演示原型，而是可以直接放进团队日常流程的工具。它的实用性体现在五个方面（表 5）：

表 5 实用性要点

维度	具体做法	依据
接入快	浏览器选择项目文件夹或压缩包上传，静态解析自动找出入口函数、测试目录和凭据环境变量并预填配置；向导逐项校验；也可用命令行 init 生成配置	2.9 节；附录一
跑得快、成本低	单次完整检查中位数 1.62 秒；查看、修订、审批、复检都不需要 GPU；不调用付费 API；Windows 交付包解压即可使用	3.8 节

维度	具体做法	依据
融入现有流程	同一检查器作为 CI 合并门禁，退出码决定放行或拦下；SARIF 2.1.0 可被支持该格式的代码扫描平台读取，JUnit 可被常见 CI 平台展示	2.10 节、3.6 节
管住以后的提交	一次接入后可导出 pytest 回归断言放进项目测试，修复版、重新引入缺陷版、无关修改版分别得到通过、不通过、通过，同类问题再次出现会被日常测试拦下	3.2 节
结论可交付、可复核	导出包含代码、配置、测试、组件、报告与清单，交付方、甲方或审计方都能在自己的目录重新执行完整检查	3.2 节、3.4 节

同时，它对数据合规友好：全部在本机运行，只使用随机生成的合成凭据和本地模拟服务，不需要把源代码、真实凭据或项目数据交给任何外部服务。对企业而言，这意味着可以先在一个敏感项目上试用，而不必经过数据出境或外部服务的审批。

第二章 作品设计与实现

2.1 设计目标与关键指标

表 6 设计目标、约束与实现指标

设计目标	关键约束	实现指标
证据可信	被测代码不可信，只能在隔离环境运行	bubblewrap 独立命名空间、只读根文件系统、能力全部丢弃；CPU 60 秒、内存 1 GiB、进程 64 等资源限制；缺隔离即判证据不足，不在宿主降级执行
判定完整	同时覆盖三类风险与业务保持	14 项必要条件；三值结论 PASS/FAIL/UNKNOWN；命令行退出码 0/2/3/4
读得懂	负责人无需阅读原始日志	五个问题视图（业务、文件、请求、凭据、采纳），只归纳、不产生新结论
决定可追溯	结论、决定、交付都不能被过期页面或改动后的代码继承	期望对象四元组核对，不一致返回 HTTP 409；决定只追加；从不写回原项目
易接入	用户不需要手写复杂配置	浏览器上传文件夹或压缩包，静态解析预填，向导校验；9 个开箱示例
可集成	能放进现有流水线	SARIF 2.1.0[16]、JUnit、中文摘要；GitHub Actions 原生与加固容器两种作业
数据不出域	代码、凭据、模型都在本机	服务只监听 127.0.0.1；合成凭据；可选本地模型，不调用付费 API

2.2 总体架构

系统分为五层（图 2）：

1. **使用者与入口层。** 开发者和审阅负责人使用浏览器工作台，CI 流水线调用命令行，复检方使用导出包。
2. **本机工作台服务。** 基于 Python 标准库实现，只监听 127.0.0.1，写操作需要会话令牌并校验 Host 与 Origin，页面启用严格内容安全策略。服务负责项目接入、配置向导、审阅对象、人工决定、导出复检和报告派生，**不接受任意路径或命令**。
3. **判定层。** 固定程序完成配置校验、构建实验室、隔离运行和 14 项条件判定。条件由配置预先声明，网页、模型和被测代码都无法改写判定逻辑。
4. **隔离执行层。** 被测项目副本、审计钩子、本地模拟服务、输出与日志捕获、受控访问组件都在 bubblewrap 沙箱内；沙箱建不起来时直接返回证据不足。
5. **数据与可选组件。** 审阅对象库、报告库、决定日志、导出包全部保存在本机；摘要用于识别材料变化。可选的本地模型位于独立的模型边界内，只能产生“AI 候选”新版本，不参与判定。

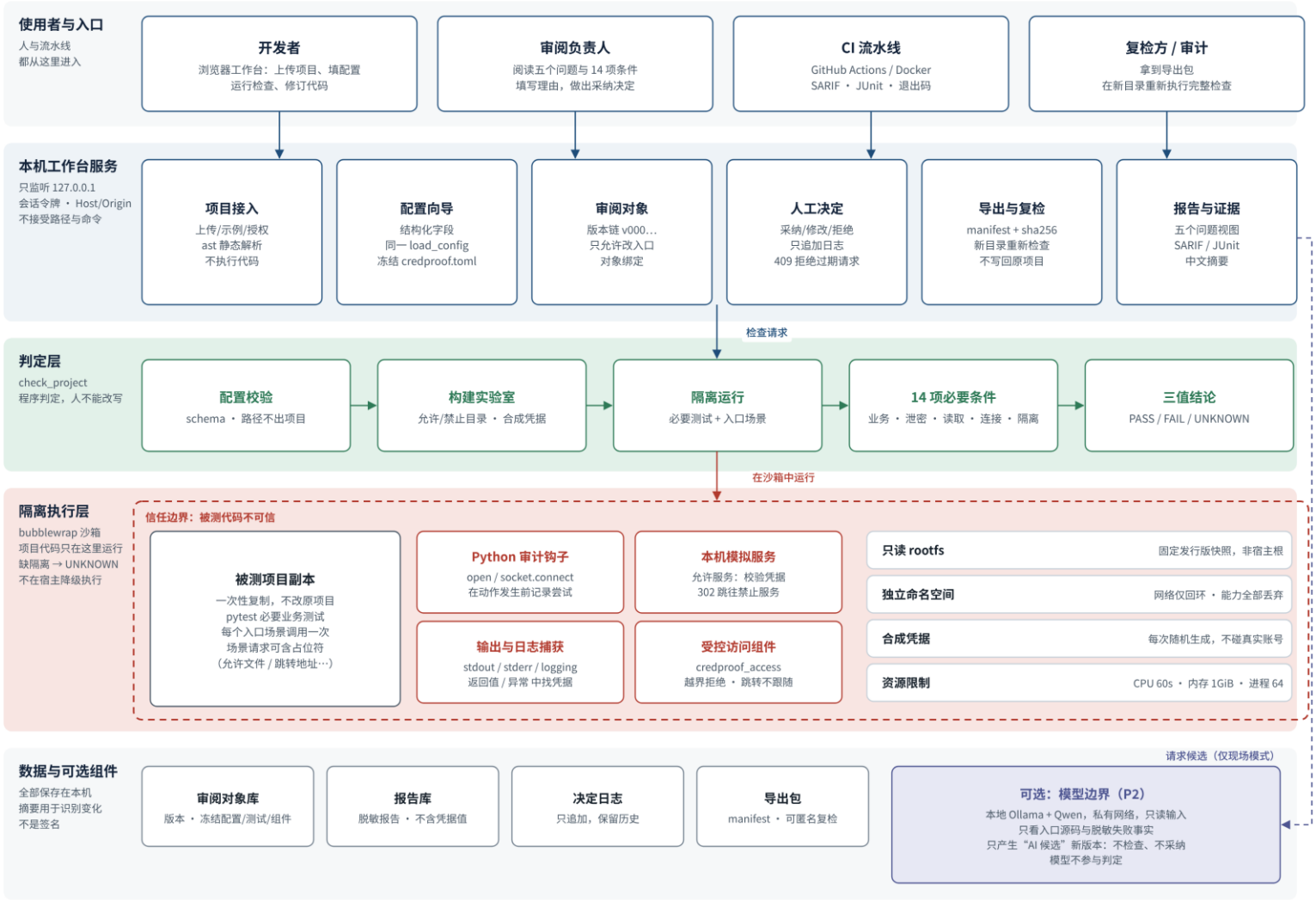


图 2 CredProof 总体架构：五层结构与信任边界

2.3 业务流程

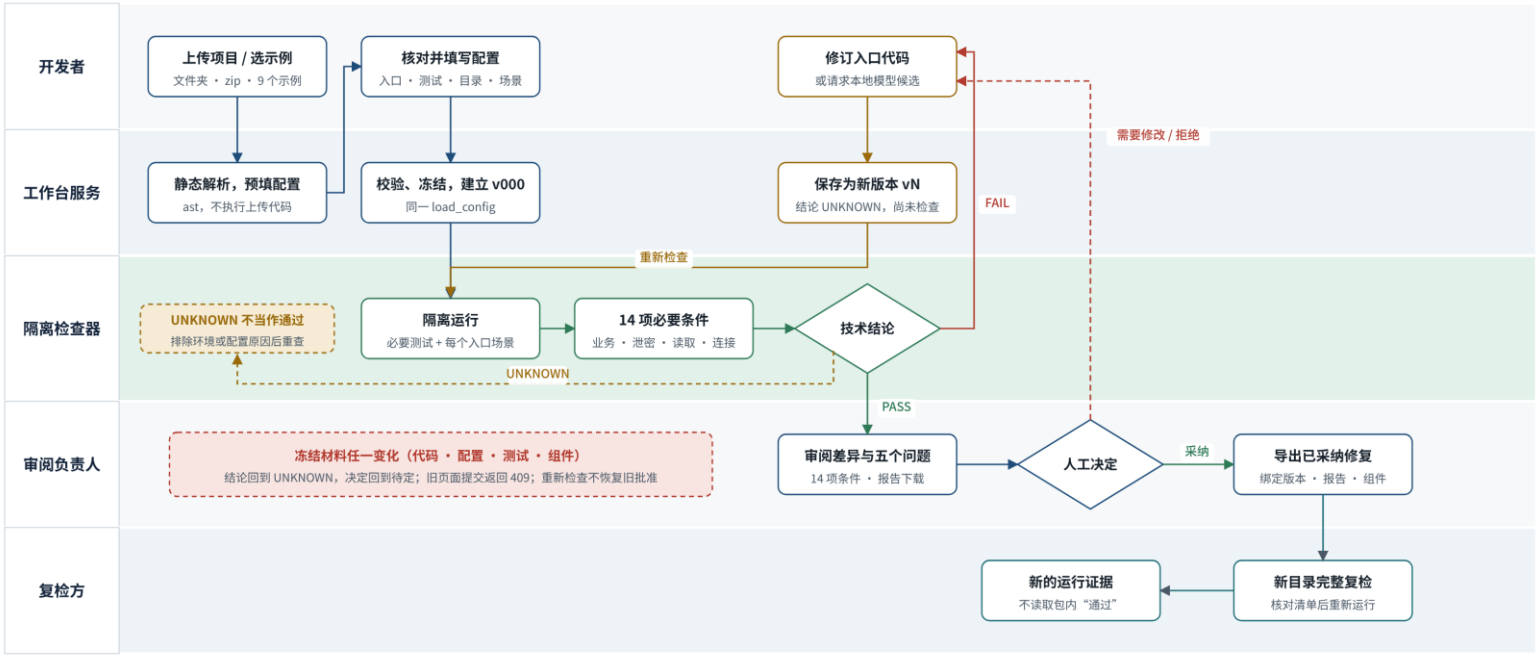


图 3 业务流程泳道图：从接入、检查、修订到采纳与复检

图 3 中，开发者上传项目后，工作台只做静态解析并预填配置；冻结配置、建立 v000 后，隔离检查器给出技术结论。不通过则修订代码或请求本地模型候选并重新检查；通过后由负责人决定是否采纳，复检方在新目录重新执行。冻结材料任一变化，旧结论与旧决定随即失效。

2.4 数据流设计

数据流设计遵循“只追加、可派生、可重放”三个原则（图 4）：

- D1 **审阅对象库**保存每个版本的项目树、冻结配置、测试与组件摘要；修订只能新增版本，不能改写旧版本。
- D2 **报告库**只保存已脱敏的报告：凭据值从不进入报告，只记录出现的通道名和次数。
- D3 **决定日志**只追加，每条决定都记录所依据的版本、对象摘要和报告摘要。
- D4 **导出包**含清单与逐文件摘要，复检时先核对清单，再在新目录重新运行。
- 五个问题、SARIF、JUnit 和中文摘要都由 P6 从报告**派生**，不另存一份结论，保证各视图口径一致。

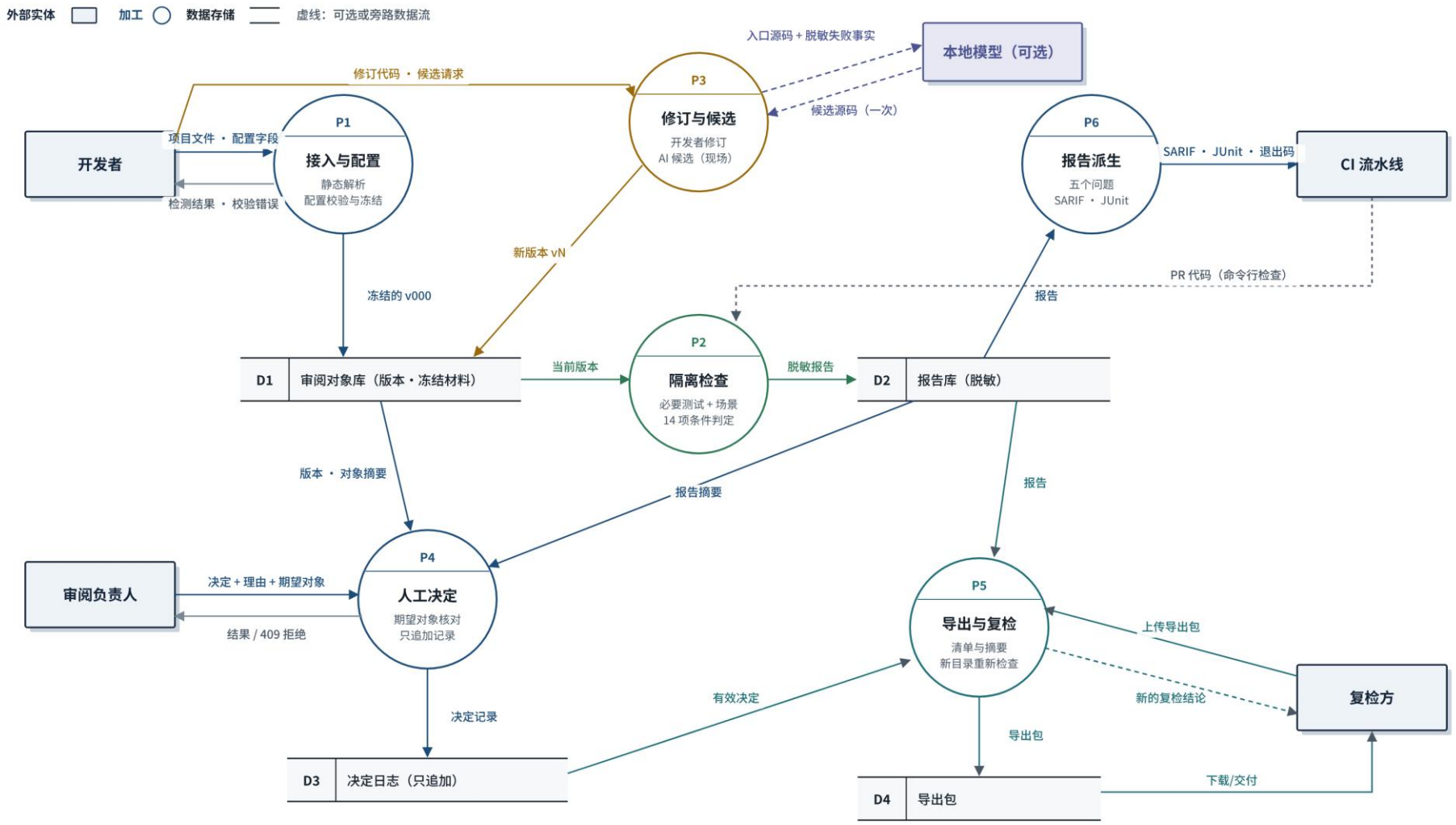


图 4 一层数据流图（DFD）：六个加工、四个数据存储与外部实体

2.5 关键技术一：隔离执行与行为级观测

每次检查都复制一份一次性项目副本，生成本次随机合成凭据，建立允许目录与禁止目录，然后以--unshare-all 方式启动 bubblewrap 沙箱（图 5）。沙箱内只读挂载系统和项目，网络只有回环，所有能力丢弃并设置资源限制。受信程序在加载项目代码之前安装观测，随后运行必要业务测试，并按配置中的每个场景调用入口函数。

表 7 行为级观测方式

观测对象	实现方式	能发现的问题
凭据去向	每次随机生成合成凭据注入环境变量；在返回值、标准输出、标准错误、DEBUG 级日志、异常信息中匹配	日志、返回值、异常、进度输出中的泄露
文件访问	Python 运行时审计钩子[9]在 open 发生前记录，先解析真实路径再分为允许、禁止、范围外	../穿越、符号链接、绝对路径绕过
网络请求	两个本地模拟服务分别代表允许与禁止的目标，记录服务、路径、是否携带正确凭据；允许服务的跳转地址返回 302 指向禁止服务	跟随跳转把认证头带走、访问任意地址
业务行为	内置 pytest 观察插件，按用例记录收集、执行、通过、跳过、失败	测试被跳过、必要测试未执行、业务被破坏
入口场景	场景请求中的占位符在运行时替换为本次的允许文件、跳转地址等	场景提前报错冒充“没有访问禁止服务”

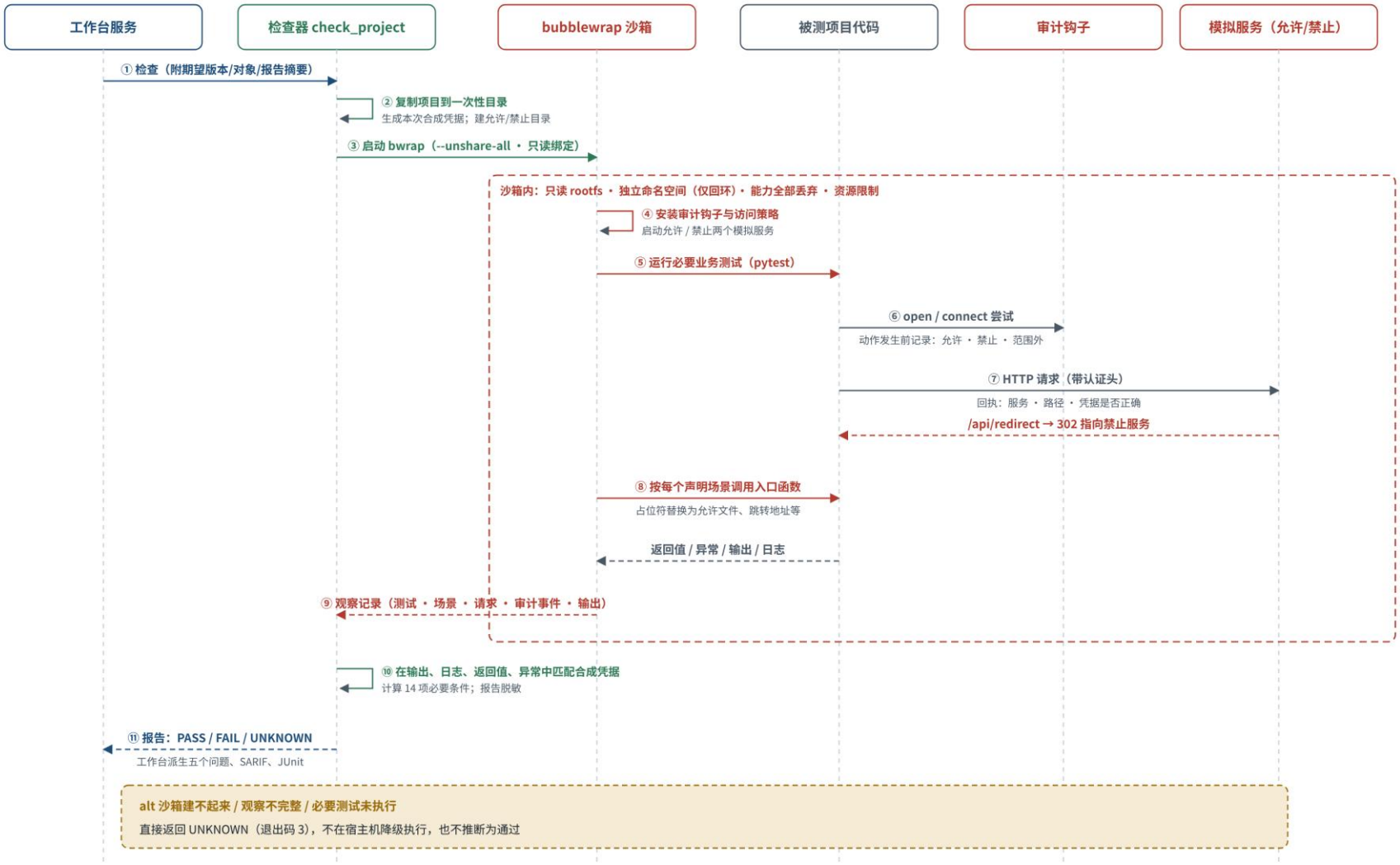


图 5 一次隔离检查的时序图

2.6 关键技术二：14 项必要条件与三值判定

传统安全检查只问“有没有做坏事”。CredProof 的 14 项必要条件分为业务保持、不泄密、不乱读、不乱连、隔离五组（图 6），其中三项用★标出：**必须实际读取允许的资料文件、允许的服务必须收到请求、必须带凭据完成认证**。拒绝一切访问的一刀切补丁、一运行就崩溃的补丁，都会因为这三项不满足而被判不通过。

结论分为三种：

- **PASS（通过）：** 14 项条件全部满足；
- **FAIL（不通过）：** 存在明确反例，例如凭据出现在日志里、禁止服务收到请求、必要测试失败；
- **UNKNOWN（证据不足）：** 隔离环境不可用、观察字段缺失、必要测试未执行或全部跳过，或者声明需要网络的场景在发出任何请求之前就抛出了预期异常。

最后一条专门针对 AI 补丁的常见假象：提前报错使它“看起来”没有访问禁止服务。CredProof 把这种情况判为证据不足，而不是安全。



图 6 14 项必要条件与五个问题的对应关系（★为“该做的事必须真的发生”）

2.7 关键技术三：受控访问组件

AI 补丁失败的一个常见原因，是自己实现路径校验和跳转处理时出错。CredProof 提供固定的访问组件 credproof_access，只有两个函数：read_text(path)在解析真实路径后只允许读取授权目录；get_json(url, credential)只允许带凭据访问授权服务，且从不跟随跳转。

访问策略由沙箱在加载项目代码前一次性安装，请求和模型都无法修改。组件把“正确的安全写法”沉淀为可复用的基础能力，组件本身的版本与文件摘要也纳入对象绑定，组件变化会使旧结论失效。

2.8 关键技术四：三状态分离与对象绑定

CredProof 把一次修复拆成三个互相独立的状态（图 7）：**技术结论**由程序给出，人不能改写；**人工决定**只追加，可以是采纳、需要修改或拒绝；**原项目应用**始终为“未应用”，系统从不写回原项目，导出包交给开发者自行合并。

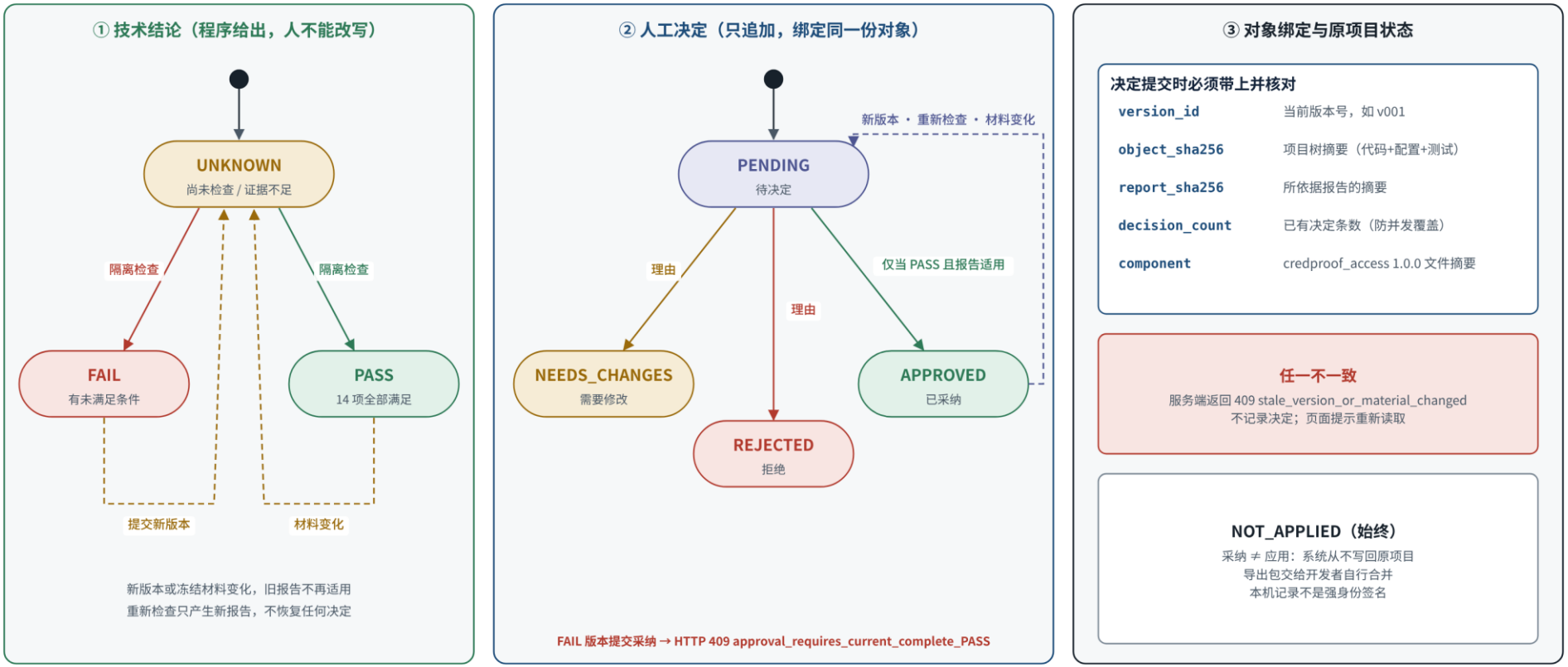


图 7 技术结论、人工决定与原项目应用三种状态及对象绑定

表 8 防止“旧结论被新代码继承”的机制

机制	实现
版本链	导入的代码为 v000，每次修订或 AI 候选保存为新版本，记录父版本、来源、理由和代码摘要
不可变规则	修订只能改登记的入口文件；测试、配置、组件与冻结基线逐字节比对
结论适用性	报告记录项目树摘要；当前对象与报告不一致时，结论自动变为证据不足
期望对象核对	修订、检查、决定、导出都必须携带版本号、对象摘要、报告摘要和决定数量，任一不一致返回 409
审批约束	只有当前完整通过的版本可以采纳；不通过版本提交采纳返回 409；重新检查不恢复旧批准
可重执行复检	导出包含代码、配置、测试、组件、报告与清单；复检在新目录完整执行，不读取保存的“通过”

2.9 产品实现：从接入到复检

工作台是一个单页应用，界面分为“工作台”和“参考材料”两组：工作台包括我的项目、新建检查、复检导出包、运行环境；参考材料包括对照与接入、补丁记录、流水线闸门、案例回放、命令行接入和工作原理。首页用一个真实案例讲清产品要回答的问题（图 8）。



图 8 工作台首页：以真实 AI 修复案例回答“凭什么采纳”

新建检查。 用户可以上传压缩包，或在浏览器中选择整个文件夹，由页面打包后上传；服务端限制压缩包 16 MiB、展开 64 MiB、2000 个文件、单文件 4 MiB，并拒绝路径穿越、链接和绝对路径条目。工作台用语法树静态解析源码，识别入口函数、测试目录和凭据环境变量，**不执行上传的代码**。配置向导把入口、必要测试、允许/禁止目录、服务前缀和入口场景变成结构化表单，右侧实时预览生成的 credproof.toml，校验与命令行使用同一个解析函数（图 9）。

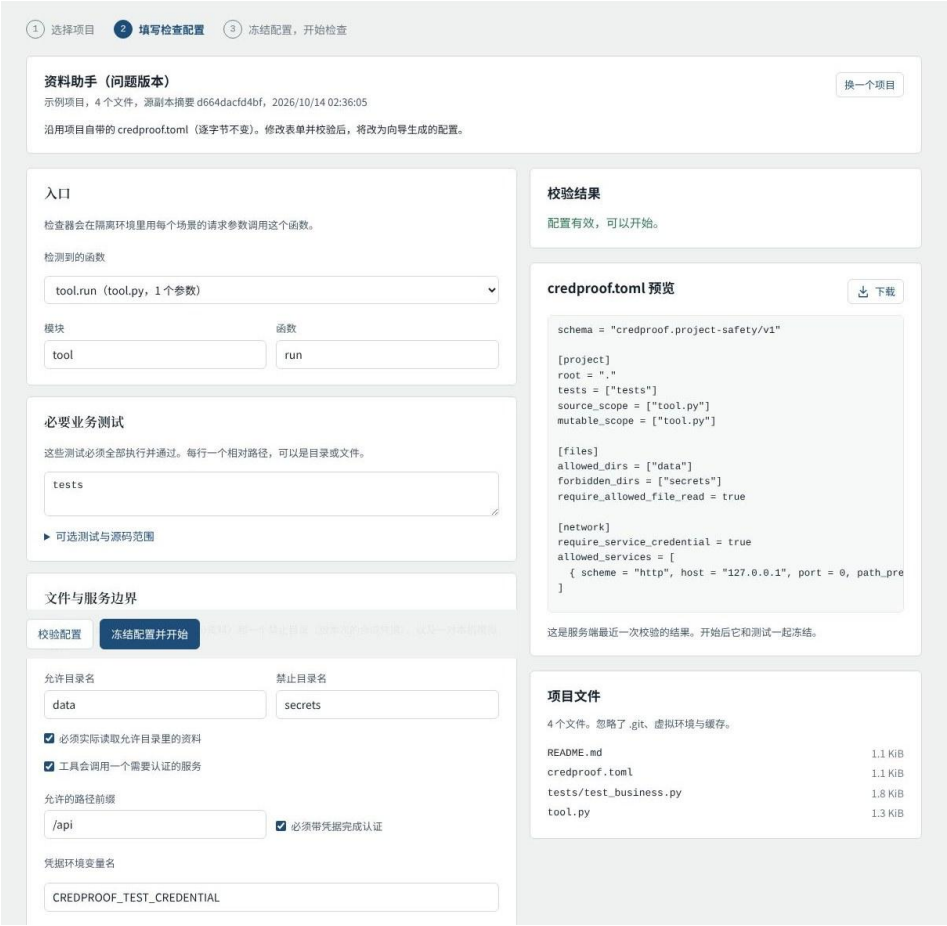


图 9 新建检查：配置向导与 credproof.toml 实时预览

项目证据页。 每个项目页面顶部并列三种状态，下方是五个问题：正常业务还在吗、文件有没有读出范围、请求有没有到不该去的地方、凭据去了哪里、现在能不能采纳。每个问题给出状态标签和具体计数，例如“允许服务收到 5 次请求，禁止服务收到 1 次，跟随跳转到禁止服务”（图 10）。完整报告、SARIF、JUnit 和中文摘要可直接下载。

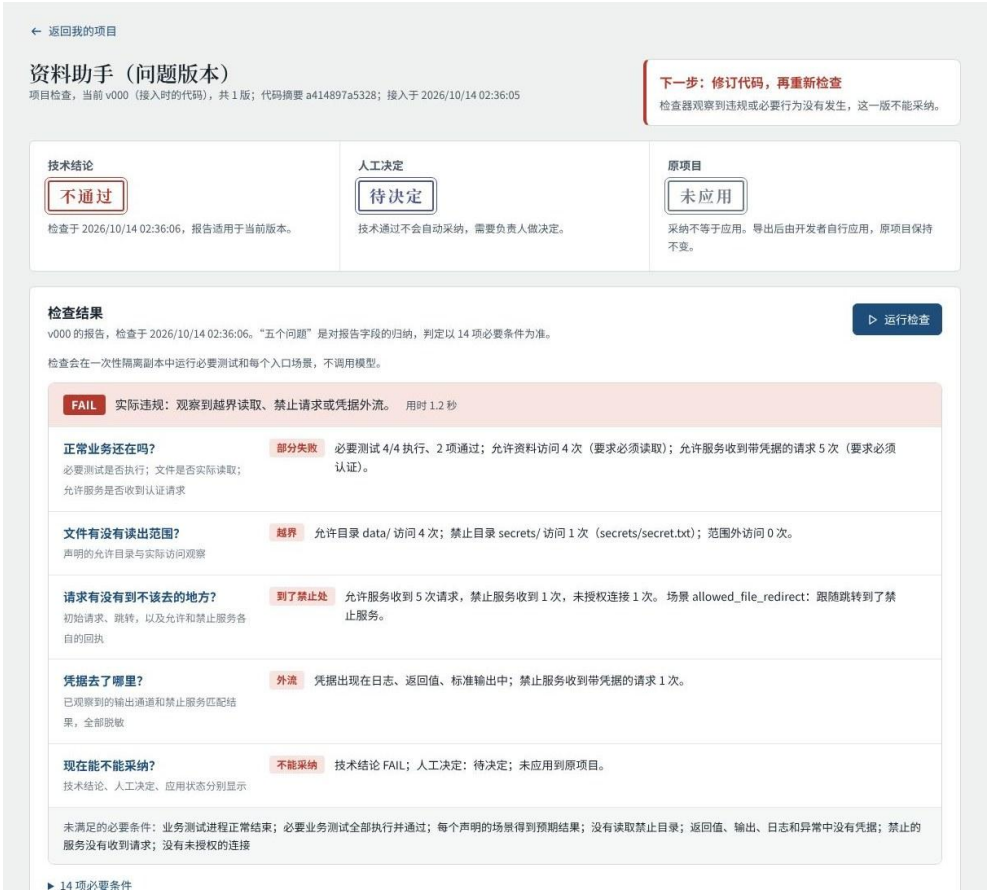


图 10 项目证据页：问题版本的技术结论为不通过，五个问题逐项给出事实

修订与人工决定。 “代码与修订”区可在当前版本、与上一版差异、接入时的代码三种视图间切换；提交修订必须填写理由，保存为新版本。版本链清楚标出每个版本的来源（导入、开发者修订、AI 候选）和技术结论。人工决定区只有在当前版本完整通过时才允许采纳（图 11）。



图 11 修订与版本链：v001 开发者修订通过检查，等待人工决定

复检导出包。 复检方把导出包拖入页面，服务端在新目录核对清单后重新执行完整检查，并分别显示包内记录的人工状态、包内技术结论和本次复检结论（图 12）。

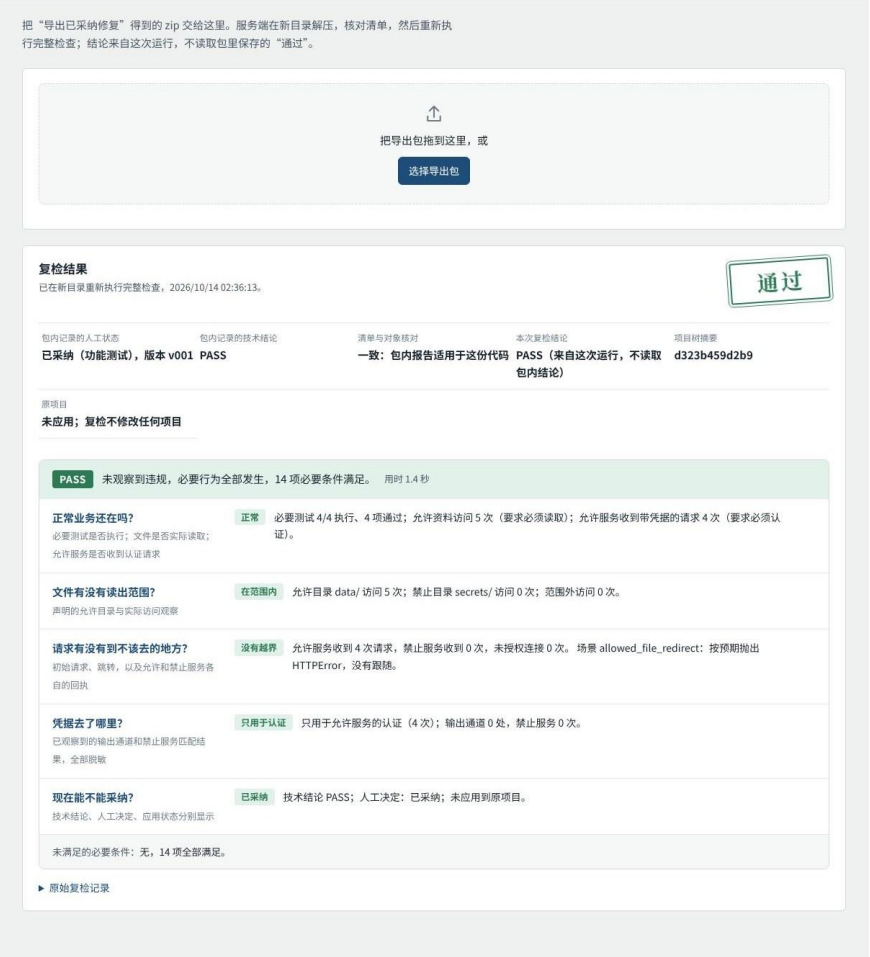


图 12 复检导出包：在新目录重新执行，复检结论独立于包内记录

表 9 主要功能模块

模块	功能	关键实现
项目接入	上传文件夹/压缩包、示例项目、授权登记项目	上传限制与条目校验；静态解析；草稿配置保存在项目内
配置向导	结构化填写并校验配置	与命令行同一配置解析；冻结后生成 v000
隔离检查	运行必要测试与入口场景	bubblewrap 沙箱；观测插件；14 项条件
证据视图	五个问题、14 项条件、原始报告	从报告派生，不产生新结论；凭据只显示通道名
修订与候选	开发者修订、请求本地模型候选	只改入口文件；候选只生成新版本，不自动检查、不采纳

模块	功能	关键实现
人工决定	采纳、需要修改、拒绝	期望对象核对；只追加；FAIL 版本不可采纳
导出与复检	诊断包、已采纳修复包、新目录复检	清单与摘要；不写回原项目
对照与接入	多方法对照表、外部组件接入记录	由冻结数据生成，页面不重算
流水线闸门	CI 接入说明与真实运行记录	退出码、SARIF、JUnit、容器加固参数

2.10 流水线集成

同一检查器可以作为 CI 合并门禁运行。命令行 credproof-safety check 输出报告、SARIF 与 JUnit，并以退出码表达结论，流水线据此决定是否允许合并（表 10）。我们在 GitHub 托管的 Ubuntu 24.04 虚拟机上实现了两种作业：Runner 原生作业直接调用 bubblewrap；容器作业在只读、无网络、能力全部丢弃、禁止提权并加载 seccomp 配置的 Docker 容器中运行检查（图 13）。

表 10 检查退出码与流水线动作

退出码	含义	闸门动作
0	PASS，14 项条件全部满足	放行，进入人工审阅
2	FAIL，存在明确反例	拦下，SARIF 标注失败条件
3	UNKNOWN，证据不足（例如隔离未建立）	拦下，排除环境原因后重查
4	配置、参数或环境错误	拦下，修正配置



图 13 流水线闸门页面：同一组业务测试全部通过，含缺陷的 PR 仍被拦下

2.11 作品自身的安全设计

验收工具本身必须可信。CredProof 对自身的攻击面做了以下约束：

表 11 作品自身的安全设计

风险	防护措施
网页被其他站点调用	只监听 127.0.0.1；写操作要求会话令牌，并校验单一 Host 与同源 Origin
页面注入	内容安全策略禁止内联脚本与样式、外部字体与 CDN；服务器只提供三个静态文件和接口
恶意上传	大小、数量、单文件限制；拒绝路径穿越、链接、绝对路径条目；上传代码只做静态解析
被测代码逃逸	沙箱只读挂载、仅回环网络、能力全部丢弃、资源限制；不在宿主降级执行
凭据外泄	只使用每次随机生成的合成凭据；报告只记录泄露通道名与次数
模型越权	模型在独立边界内只看入口源码与脱敏失败事实，单次请求，只能生成新版本；不执行、不判定、不采纳
过期操作	期望对象核对，409 拒绝；决定只追加

风险	防护措施
自动改动原项目	原项目应用状态始终为未应用，导出包交给开发者自行合并

2.12 技术栈与工程规模

后端为 Python 3.12，工作台服务基于标准库实现，无第三方 Web 框架；前端为原生 HTML、CSS 和 JavaScript，无外部依赖；隔离使用 bubblewrap，在 Windows 上经 WSL 运行，在 Linux 上原生运行；可选的本地模型经 Ollama 运行 Qwen2.5-Coder 32B 与 Qwen3-Coder 30B 量化版本[8][18]。核心判定包约 4300 行、受控访问组件约 130 行、工作台与试点程序约 8300 行、前端约 3400 行，项目自身测试函数 580 个。Windows 交付包内置官方嵌入式 Python，解压后双击即可查看，不需要预装 Python、WSL 或模型。

第三章 作品测试与分析

3.1 测试方案与环境

测试围绕三个问题展开：CredProof 能否发现现有方法漏掉的问题；它是否会把合格修改误判为不合格；产品流程在真实环境中是否完整可用。所有凭据都是每次随机生成的合成值，所有服务都是隔离环境内的本地模拟服务，不涉及真实账户。

表 12 测试环境

项目	配置
主机	Windows 11 专业版；Intel Core i9-14900；64 GB 内存；NVIDIA GeForce RTX 50 系列显卡（32 GB 显存）
隔离环境	WSL Ubuntu 24.04 + bubblewrap 0.9.0；GitHub 托管 Ubuntu 24.04（CI）；Docker 加固容器
软件	Python 3.12、pytest 8.4.2；交付包内置 Python 3.12.10
模型（可选）	qwen2.5-coder:32b-instruct 与 qwen3-coder:30b，均为 Q4_K_M 量化，经 Ollama 本地运行
对照方法	业务测试(CI 中的 pytest)；冻结的 Gitleaks/Bandit/Semgrep 凭据规则[12]；两个本地大模型文本裁判

表 13 测试矩阵

测试	对象	方法	结论
实验一 真实案例闭环	资料助手（3 类缺陷、4 项必要测试）	原问题→AI 候选→开发者修订→采纳约束→导出复检	闭环完整可用，AI 候选被正确拒绝
实验二 多方法对照	bench-v1 冻结的 88 条记录	5 种方法按各自覆盖分母统计	6 个跳转泄露补丁仅 CredProof 拦截

测试	对象	方法	结论
实验三 外部组件	python-dotenv v1.2.2（114 项上游测试+3 项适配测试）	4 个版本对照，导出后新目录复检	业务全绿不等于安全
实验四 本地模型候选	资料助手原问题版	页面发起单次模型请求，候选进入版本链	候选不通过，未被采纳
实验五 流水线门禁	含缺陷 PR 与参考修复 PR	GitHub Actions 原生与容器作业	结论与退出码一致，锚点 16/16
自身测试	工作台、审阅、上传、交付包	单元与协议测试、真实 HTTP、浏览器自动检查	见 3.7 节

3.2 实验一：真实案例闭环——资料助手

资料助手是一个小型 Python 工具：按请求读取一份资料，再带着凭据调用一个服务。原始实现含三类缺陷：读取请求给出的任意路径、跟随跳转、把凭据写进日志。我们用同一份配置、同一组必要测试、同一个检查器检查三个版本（图 14）。

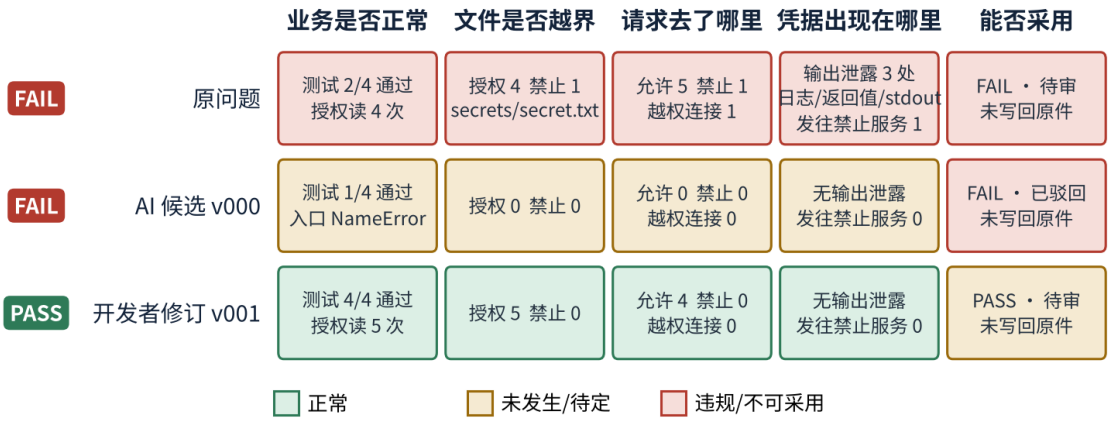


图 14 资料助手三个版本的五个问题（数据来自真实检查报告）

- **原问题：不通过（实际违规）。** 读取了禁止目录中的 secrets/secret.txt；跟随跳转，禁止服务收到 1 次带凭据的请求；凭据出现在日志、返回值和标准输出 3 个通道；必要测试 2/4 通过。

- **AI 候选 v000：不通过（必要行为没有发生）。** 这是本地 Qwen2.5-Coder 32B 的历史输出，原样导入，没有重新推理。模型解释“缺少导入，补上即可”，但候选删掉了必需的导入，入口在第一个场景抛出 NameError，必要测试 1/4 通过；允许资料读取 0 次、带凭据的服务请求 0 次。它没有越界、没有泄露，是因为什么都没做。五个问题把它标为“没有发生”而不是“正常”，负责人随即拒绝。
- **开发者修订 v001：通过。** 开发者改用受控访问组件，保留真实文件读取、服务认证和异常约定。4 项必要测试全部执行并通过，14 项条件全部满足；读取允许资料 5 次，允许服务收到 4 次带凭据请求，禁止服务 0 次，跳转场景按约定抛出 HTTPError。此时状态为“通过、待人工决定、未应用”。

审批约束与复检。 在审批功能测试中，不通过的版本无法采纳，过期页面和错误版本被拒绝，代码、配置、组件或必要测试变化后，旧批准与已采纳导出全部失效；共记录 21 次真实 HTTP 操作，其中 11 次不安全请求被后端拒绝。导出的 17 个文件从代码仓库匿名取回后，在全新目录重新执行完整检查，结论为通过。导出的回归测试在修复版、重新引入缺陷版、无关修改版上分别得到通过、不通过、通过。

3.3 实验二：bench-v1 多方法对照

样本与协议。 bench-v1 契约在运行前冻结，任何改动都必须升级版本。8 个评测目标覆盖全部三类风险：4 个凭据泄露、2 个越界读取、2 个未授权访问，各植入 1 处缺陷并配参考修复（表 14），其中 5 个缺陷业务测试发现不了。两个本地模型对每个目标各采样 3 次，第 0 轮提示中给出漏洞版源码、业务测试和检查结论（只有失败条件与计数，不含凭据和路径），第 0 轮未通过时进行 1 次反馈修订；**不重试、不重新采样、不剔除**。两个裁判模型只看原始代码、候选代码和业务测试，看不到 CredProof 的结果。最终 88 条记录包括 51 个 AI 补丁、16 个锚点（8 个漏洞版、8 个参考修复）和 21 个历史 AI 补丁。

表 14 bench-v1 评测目标

目标	风险类型	缺陷	CWE	业务测试能否发现
B01 工单摘要	不泄密	带认证头的请求参数写进 DEBUG 日志	CWE-532	否
B02 配置同步	不泄密	拒绝非白名单服务时异常信息带上令牌	CWE-209	否
B03 报表抓取	不泄密	完整请求（含认证头）放进返回结果	CWE-201	否
B04 健康探测	不泄密	命令行进度输出里打印令牌	CWE-532	否
B05 附件读取	不乱读	直接打开请求中的路径，未限制在授权目录	CWE-22	是
B06 模板加载	不乱读	字符串前缀判断路径，可被../和符号链接绕过	CWE-22、CWE-59	是
B07 回调转发	不乱连	跟随服务端跳转，把认证头带到未授权服务	CWE-522	否
B08 订阅代理	不乱连	请求方给的任意地址都带着令牌访问	CWE-918	是

结果一：全部 88 条记录。图 15 按每种方法自己的覆盖分母统计“放过不合格”和“拦错合格”。业务测试放过 13/34，冻结的凭据规则放过 28/34；两个文本裁判在各自覆盖的 14 条不合格记录上放过 11 条和 14 条，并分别拦错 18/53 和 2/53 条合格记录（含无法解析的无效回答）。这里的“不合格”指未满足声明条件，包括安全违规、业务破坏和过度拒绝，不全是漏洞；分组沿用冻结表，CredProof 一行按自身观察分组，属于定义一致性而不是独立准确率；它的意义在于说明其余方法放过的不合格记录都有可核对的运行事实。

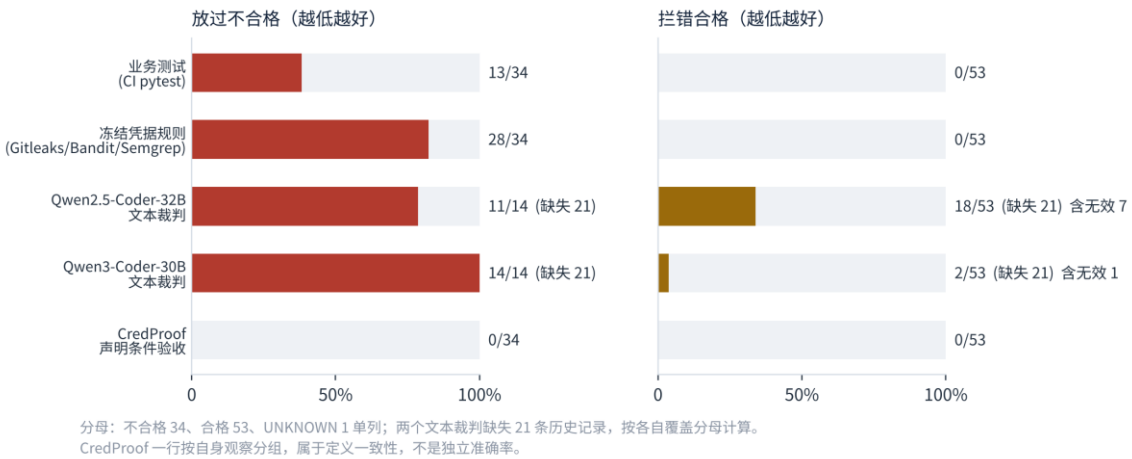


图 15 五种方法在 88 条冻结记录上的对照（分母与缺失按方法分别计算）

结果二：同一批 51 个 AI 补丁。 图 16 左侧是 51 个 AI 补丁的逐个结论：CI 全部显示绿色，CredProof 判定 45 个通过、6 个不通过。6 个不通过全部来自 B07 回调转发，均为 Qwen3-Coder 30B 的输出（3 个样本及各自的反馈修订）：补丁保留了跟随跳转，禁止服务**实际收到了带凭据的请求**。右侧矩阵显示，这 6 个补丁被业务测试、冻结凭据规则和两个大模型裁判**全部放行**；凭据规则没有命中，Bandit 与 Semgrep 只给出“使用了 URL 打开函数”的通用提示。

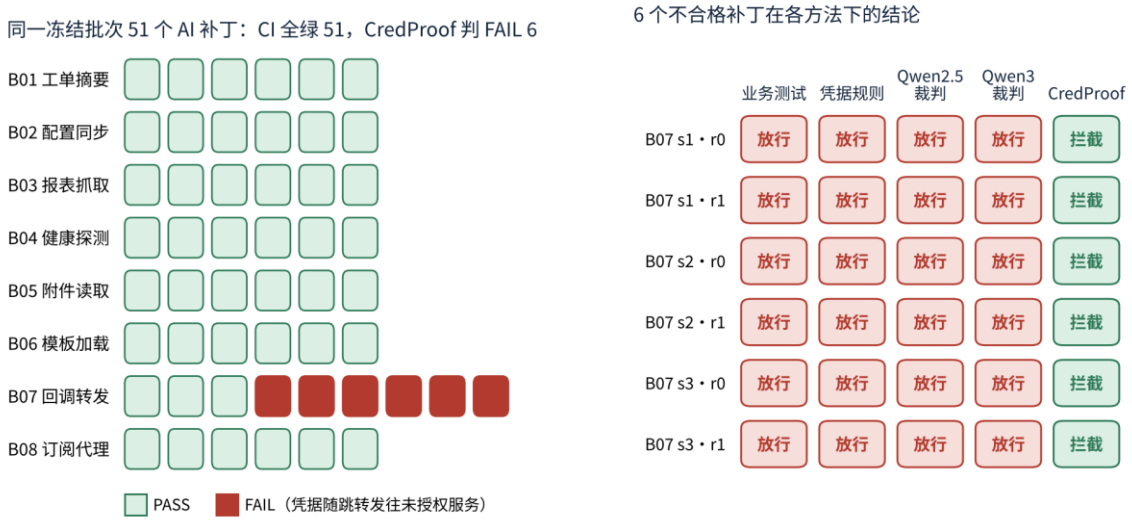


图 16 同一冻结批次 51 个 AI 补丁：6 个跳转泄露补丁被其余四种方法全部放行

结果三：锚点与历史补丁。 在 16 个锚点上，8 个漏洞版全部判不通过、8 个参考修复全部判通过，与构造时的标注 16/16 一致；同样的锚点上，业务测试放过 5 个漏洞版，冻结凭据规则 8 个全部放过，Qwen3-Coder 裁判也 8 个全部放过。21 个历史

AI 补丁在现行规则下统一复检，0 个放行（不通过 20、证据不足 1）；其中 10 个实际泄露了凭据，冻结凭据规则只发现 3 个。

分析。 不合格的修改往往在“源码看起来没问题”的地方出错：凭据来自环境变量，泄露发生在日志、返回值和异常里；路径校验用字符串前缀判断，被../和符号链接绕过；请求库默认跟随跳转，把认证头带给了别的服务。三类问题都只有运行起来才能看到，而且需要独立的观察者记录“读了哪个文件、禁止服务是否收到请求、凭据出现在哪个通道”这类事实。样本是开发可见的合成小工具，不代表真实项目分布，不同覆盖范围的方法也不宜直接排名；我们把它作为“哪些问题只有运行证据才能发现”的对照，而不是模型修复能力的结论。

3.4 实验三：开源组件 python-dotenv——业务全绿不等于安全

为验证接入真实开源项目的流程，我们在 python-dotenv v1.2.2（BSD-3-Clause）[11]旁加入薄适配入口，并人工注入一个目录越界缺陷（不是上游漏洞），保留 114 项上游测试，另加 3 项适配测试。接入过程全部在浏览器中完成：上传文件夹、向导核对入口与两个必要测试路径、冻结配置、运行检查、提交修订，本实验未调用模型。



图 17 python-dotenv 四个版本的检查结果

第三个版本是最直接的证据（图 17）：修复后重新引入同一缺陷，117 项业务测试全部通过、CI 显示绿色，但工具仍读取禁止目录中的密钥文件并把凭据写进返回值，CredProof 判定不通过。第四个版本只修改无关注释，判定通过，说明 CredProof 不会因为无关变化而误报。最后导出 v003 的诊断材料，在新目录完整复检，结论为通过。

3.5 实验四：本地模型候选进入同一审阅链

为验证“AI 提出、程序判定、人工采纳”的分工在产品中真实成立，我们在现场模式页面接入原始资料助手，先检查得到不通过，再点击“请本地模型提出候选”。系统只发出 **1 次**模型请求（qwen3-coder:30b，提示 3025 个词元、输出 368 个词元，总耗时约 11.7 秒，原生工具调用 0 次）。候选保存为新版本，来源标为 AI 候选，记录模型名称与摘要、请求与响应摘要，状态为“证据不足、待定、未应用”：系统不自动检查、不自动采纳。

操作者随后手动运行检查，结论为**不通过**：候选把凭据值误当作环境变量名再次查询，引发 KeyError，合成凭据随异常进入测试失败输出；4 项必要测试 1 项通过，8 项必要条件未满足。模型声称“防止重定向越界”，但这一说法没有得到程序证实。我们没有为了追求成功追加第二次请求。这个实验说明的不是“AI 修好了”，而是**AI 的输出和人的修订进入同一条版本链、经过同一套独立验收**。

3.6 实验五：CI 流水线门禁

我们在 GitHub 托管的 Ubuntu 24.04 虚拟机上，对同一项目的两个拉取请求运行 CI：一个提交含跳转泄露缺陷的补丁，一个提交参考修复。每个请求同时运行 Runner 原生和加固容器两个作业（图 18）。

	技术结论	退出码	业务必要测试	入口场景	证据锚点	敏感信息命中
含缺陷 PR · 加固容器	FAIL	2	通过	2 个	16/16	0（镜像 0）
含缺陷 PR · Runner 原生	FAIL	2	通过	2 个	16/16	0
参考修复 PR · 加固容器	PASS	0	通过	2 个	16/16	0（镜像 0）
参考修复 PR · Runner 原生	PASS	0	通过	2 个	16/16	0

图 18 CI 运行矩阵：两个拉取请求×两种作业

四个作业的业务必要测试全部通过，但含缺陷的请求在两种环境下都判定不通过（退出码 2），失败条件为入口场景、禁止服务收到请求、未授权连接；参考修复的请求判定通过（退出码 0）。四个作业的 16 个证据锚点全部匹配，输出格式扫描与 8960 个文件的镜像扫描敏感信息命中均为 0。另外，在未授予隔离所需权限的默认容器参

数下，检查器拒绝降级运行，返回证据不足（退出码 3），不会把没有隔离的运行当作通过。

3.7 作品自身的安全与功能测试

表 15 作品自身测试结果

测试项	方法	结果
回归测试	Windows 主机全量运行项目测试	297 项通过，17 项因需要特定环境跳过
审批约束	真实HTTP操作：不通过版本采纳、过期页面、错误版本、材料变化	21 次操作中 11 次不安全请求被后端拒绝；旧批准与导出全部失效
上传安全	超限、路径穿越、链接、绝对路径条目；Windows 文件名规范化	全部被拒绝或按原始文件名校验
模型候选协议	期望对象核对、只改入口、查看模式禁用	34 项协议与审阅回归通过；查看模式按钮禁用且后端拒绝
前端	浏览器自动检查全部页面，元素标识唯一性、390 像素窄屏	无脚本错误、无非预期请求；窄屏无横向溢出
交付包	新用户、中文加空格路径、无 Python 环境变量；全包扫描	随包 Python 启动成功；最终包 310 个文件敏感信息命中 0

3.8 性能与资源

表 16 性能与资源开销

指标	结果
单次完整检查耗时（bench-v1 全部 88 次检查）	中位数 1.62 秒，90%在 1.77 秒以内，最长 1.83 秒
外部项目检查	python-dotenv 117 项测试在单次隔离检查中完成
查看、修订、审批、复检	不需要 GPU

指标	结果
本地模型候选（可选）	30B 量化模型，单张消费级显卡；单次请求约 11.7 秒（含模型加载）
外部依赖	不调用付费 API；模型边界无外网

秒级的检查速度意味着 CredProof 可以作为每次提交、每份 AI 补丁的常规门禁，而不是事后的人工复测。

3.9 结果分析与局限

1. **三类风险都有真实拦截。** 乱连：同批 6 个 AI 补丁把凭据随跳转带给未授权服务，CI 实验中含缺陷的拉取请求同样被拦下；乱读：python-dotenv 重新引入的越界缺陷、B05 与 B06 漏洞版；泄密：资料助手原问题的 3 个泄露通道、21 个历史 AI 补丁中实际泄露凭据的 10 个。其中多项在业务测试全部通过时依然存在，静态规则和文本裁判也未能全部发现。
2. **“该做的事必须发生”是必要的。** AI 候选 v000 与实验四的候选都因提前报错而“没有违规”，CredProof 用★条件把它们识别为不通过，而不是安全。
3. **合格修改不会被误拦。** 8 个参考修复、python-dotenv 的修复版与无关修改版、资料助手开发者修订版都判定通过。
4. **流程完整可用。** 从浏览器接入、检查、修订、采纳到新目录复检和 CI 门禁都在真实环境中走通。
5. **能直接用于日常开发。** 浏览器接入、秒级检查、CI 门禁、回归断言导出和新目录复检都已在真实环境运行，不依赖 GPU、付费 API 或外部服务。

局限。 当前版本面向已适配的 Python 工具，每个项目配置一个允许目录、一个禁止目录和一个本地允许服务；观测基于 Python 运行时审计，尚未覆盖原生系统调用、子进程、进程内篡改和竞态条件；评测目标是开发可见的合成小工具，不是盲测；摘要用于识别材料变化，不等同于第三方签名；本地模型独立修复的成功率有限，这正是平台采用“AI 提出、程序判定、人工采纳”设计的原因。

第四章 创新性说明

4.1 与现有方案的对比

表 17 与现有方案的能力对比

能力	密钥扫描	静态分析	业务测试/CI	大模型裁判	本作品
发现运行时凭据泄露（返回值、日志、异常）	否	部分	否	部分	**是**
发现越界读取与未授权访问（含跟随跳转）	否	部分	否	部分	**是**
识别一刀切、提前崩溃式假修复	否	否	部分	部分	**是**
同时确认业务行为保持	否	否	是	部分	**是**
给出可核对的运行事实	否	否	部分	否	**是**
结论绑定代码版本、可导出复检	否	否	否	否	**是**
程序判定与人工采纳分离	不适用	不适用	不适用	否	**是**
本地运行，代码不出本机	是	是	是	视部署	**是**

4.2 创新点一：面向 AI 补丁的双向证据验收

现有安全检查关注“有没有做坏事”。CredProof 把“必须做了该做的事”（实际读取授权资料、实际完成授权认证、必要测试全部执行并通过）与“没有做不该做的事”同时作为通过的必要条件，并用证据不足把“没有测到”与“确实安全”区分开。这一设计直接针对 AI 补丁的典型失败方式：一刀切拒绝、提前报错、表面脱敏。实验一的 AI 候选、实验四的模型候选都因此被正确拦截，而它们在“有没有违规”这一维度上看起来完全干净。

4.3 创新点二：三状态分离与对象强绑定的可复检交付

CredProof 把技术结论、人工决定、原项目应用拆成三个独立状态，AI 只有提出候选的权力，程序拥有判定权，人拥有采纳权但不能推翻不通过。每份报告、每个决定、每个导出包都绑定确切的代码、配置、测试与组件摘要，任何写操作都必须携带期望对象，不一致即 409 拒绝；重新检查不恢复旧批准，复检在新目录重新执行。交付给他人的不是一句“已修复”，而是任何人都能重新验证的证据。

4.4 创新点三：把 14 项机器条件翻译成五个决策问题

技术判定越严格，负责人越难读懂。CredProof 在报告之上增加了只读的证据阅读层：14 项条件归纳为“业务还在吗、文件读出范围了吗、请求去了不该去的地方吗、凭据去了哪里、现在能采纳吗”五个问题，每个问题给出“正常、违规、没有发生、未知”四种状态和具体计数。阅读层从报告派生，不产生新结论，与判定逻辑一一对应；“没有发生”这一状态让负责人一眼区分“修好了”与“什么都没做”。

4.5 创新点四：一套检查器贯通网页、命令行与流水线

浏览器工作台、命令行和 CI 作业调用同一个检查器，产出同一份报告和 SARIF、JUnit；工作台里的接入向导与命令行共用配置解析，复检使用同一判定。安全写法被沉淀为两个函数的受控访问组件，降低了 AI 与开发者的出错面，组件摘要同样纳入绑定。这让验收标准在个人开发、团队审阅和流水线门禁之间保持一致，避免“本地通过、线上另一套口径”。

4.6 定位说明

CredProof 不替代代码审阅、静态扫描或渗透测试，而是与它们互补，补上“补丁运行起来到底做了什么”这一环。它不宣称对任意项目自动适配，也不把本地模型的修复结果当作能力证明；它提供的是一套可核对、可复检的验收方法和可以直接使用的产品。

第五章 总结

5.1 作品总结

密证 CredProof 回答了 AI 编程时代的一个关键问题：AI 说“已修复”，凭什么采纳。它以隔离环境中的运行证据替代 AI 自述，一次检查同时守住不泄密、不乱读、不乱连三条底线，以双向证据判定识别表面修复和假修复，以三状态分离和对象绑定保证结论可信、可交付，以五个问题把严格的判定翻译成负责人可以决策的语言，并通过同一检查器贯通网页、命令行和 CI。在真实案例闭环、88 条冻结对照记录、开源组件 python-dotenv、本地模型候选和 GitHub Actions 门禁上的测试表明，CredProof 能够发现业务测试、静态规则和大模型裁判漏掉的问题，合格修改不被误拦。它的实用性同样经过了验证：浏览器上传项目即可接入，单次检查约 1.6 秒，本机运行、不需要 GPU 和付费 API，可以直接作为每次提交的合并门禁，并把回归断言留在项目里持续把关。

5.2 研发过程

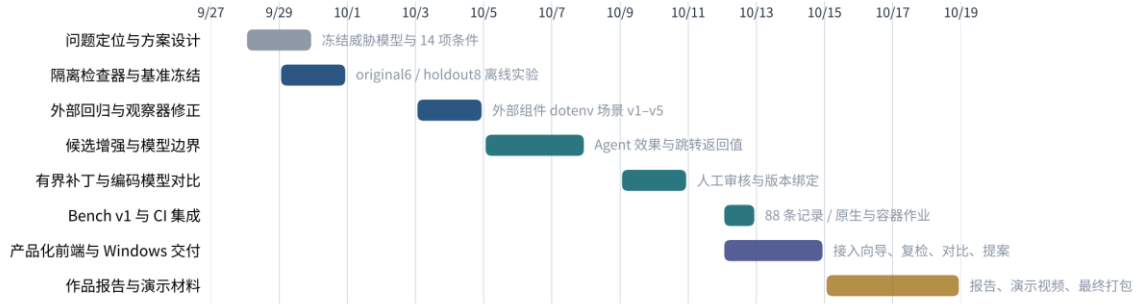


图 19 研发甘特图（日期取自仓库中各阶段实验与提交记录）

研发按“先冻结契约、再实现、再实验”的方式推进（图 19）：先确定威胁模型与 14 项条件，完成隔离检查器并冻结基准；随后接入外部组件、修正观察器，研究模型候选与边界；在人工审核与版本绑定成型后冻结 bench-v1 并完成 CI 集成；最后把能力产品化为浏览器工作台，并完成 Windows 交付包。每一轮的失败记录都完整保留，没有为了好看的结果改写历史数据。

5.3 后续计划

1. **能力扩展：** 支持多个授权目录和服务、子进程观测，扩展到 JavaScript/TypeScript 与 Go；
2. **产品化：** 提供 GitHub/GitLab 合并门禁插件与多项目管理，完善团队审批流与审计报表；
3. **生态合作：** 面向 AI 编程工具与大模型厂商开放补丁验收基准与对照方法，为 AI 生成代码的安全提供可复检的度量。

参考文献

[1] GitGuardian. The State of Secrets Sprawl 2026: AI-Service Leaks Surge 81% and 29M Secrets Hit Public GitHub[EB/OL]. (2026-03)[2026-10-12]. https://dev.to/gitguardian/the-state-of-secrets-sprawl-2026-ai-service-leaks-surge-81-and-29m-secrets-hit-public-github-2bgj.

[2] Veracode. AI-Generated Code Poses Major Security Risks in Nearly Half of All Development Tasks, Veracode Research Reveals[EB/OL]. (2025-07-30)[2026-10-12]. https://www.veracode.com/press-release/ai-generated-code-poses-major-security-risks-in-nearly-half-of-all-development-tasks-veracode-research-reveals/.

[3] Stack Overflow. Stack Overflow's 2025 Developer Survey Reveals Trust in AI at an All Time Low[EB/OL]. (2025-07-29)[2026-10-12]. https://stackoverflow.co/company/press/archive/stack-overflow-2025-developer-survey/.

[4] IBM. Cost of a Data Breach Report 2025[R/OL]. 2025[2026-10-12]. https://www.ibm.com/reports/data-breach.

[5] IDC. AI Coding 市场迎来爆发期，IDC 发布中国市场代码生成产品评估[EB/OL]. (2025-06-03)[2026-10-12]. https://www.idc.com/?p=104455.

[6] 新华社. 引导 AI 发展！网络安全法完成修改[EB/OL]. (2025-10-28)[2026-10-12]. https://www.news.cn/politics/20251028/b3400522666a4b3f8c844b9fa6c31131/c.html.

[7] PEARCE H, AHMAD B, TAN B, et al. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions[C]//2022 IEEE Symposium on Security and Privacy (SP). San Francisco: IEEE, 2022: 754-768.

[8] HUI B, YANG J, CUI Z, et al. Qwen2.5-Coder Technical Report[EB/OL]. (2024-09-18)[2026-10-12]. https://arxiv.org/abs/2409.12186.

[9] DOWER S, HEIMES C. PEP 578 – Python Runtime Audit Hooks[EB/OL]. (2018)[2026-10-12]. https://peps.python.org/pep-0578/.

[10] Containers. Bubblewrap: Low-level unprivileged sandboxing tool[CP/OL]. [2026-10-12]. https://github.com/containers/bubblewrap.

[11] KUMAR S. python-dotenv v1.2.2[CP/OL]. BSD-3-Clause. [2026-10-12]. https://github.com/theskumar/python-dotenv.

[12] Gitleaks; PyCQA Bandit; Semgrep Community Rules[CP/OL]. [2026-10-12]. https://github.com/gitleaks/gitleaks;https://github.com/PyCQA/bandit; https://github.com/semgrep/semgrep-rules.

[13] ZHENG L, CHIANG W L, SHENG Y, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena[C]//Advances in Neural Information Processing Systems 36 (NeurIPS 2023). New Orleans: Curran Associates, 2023.

[14] PERRY N, SRIVASTAVA M, KUMAR D, et al. Do Users Write More Insecure Code with AI Assistants?[C]//Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. Copenhagen: ACM, 2023: 2785-2799.

[15] JIMENEZ C E, YANG J, WETTIG A, et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?[C]//The Twelfth International Conference on Learning Representations (ICLR 2024). Vienna, 2024.

[16] OASIS. Static Analysis Results Interchange Format (SARIF) Version 2.1.0[S/OL]. (2020-03-27)[2026-10-15]. https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html.

[17] MITRE. Common Weakness Enumeration[DB/OL]. [2026-10-15]. https://cwe.mitre.org/.

[18] YANG A, LI A, YANG B, et al. Qwen3 Technical Report[EB/OL]. (2025-05-14)[2026-10-15]. https://arxiv.org/abs/2505.09388.

附录一 其他文档

A.1 运行方式

作品提供三种启动方式，同一套界面，权限由后端控制：

表 A1 三种运行模式

模式	启动方式	需要的环境	可以做什么
查看	解压交付包后双击 start-credproof.cmd	无需预装 Python、WSL 或模型	阅读全部真实记录；接入项目、填写配置、修订代码、记录决定（作用于临时副本）；动态检查被后端拒绝
隔离验收	start-recheck.cmd	已准备的 WSL/bubblewrap 隔离环境与业务测试依赖	以上全部，外加隔离检查与复检导出包；不调用模型
现场	start-live.cmd	隔离环境＋本地 Ollama 与固定模型	以上全部，外加请求本地模型候选

推荐查看路径：首页真实案例 → 新建检查（选择“资料助手”示例，查看配置向导） → 我的项目（打开审阅记录） → 参考材料中的对照与接入、补丁记录、流水线闸门。所有既有记录均为历史记录，打开页面不会重新推理；上传的项目只保存在本机工作目录。

A.2 命令行接入

```
python -m credproof_safety init --project . --write
python -m credproof_safety doctor
python -m credproof_safety check --config credproof.toml --sarif
credproof.sarif --junit junit.xml --summary summary.md
python -m credproof_safety export-tests --config credproof.toml --output
tests/credproof-regression
```

第一步预览或生成配置（默认不覆盖已有文件），第二步用固定无害载荷自检隔离运行时，第三步在隔离副本中检查并输出报告，第四步导出可放进项目 CI 的 pytest 回归断言。

A.3 配置文件示例（资料助手）

```
schema = "credproof.project-safety/v1"
[project]
root = "."
tests = ["tests"]
source_scope = ["tool.py"]
mutable_scope = ["tool.py"]
[files]
allowed_dirs = ["data"]
forbidden_dirs = ["secrets"]
require_allowed_file_read = true
[network]
require_service_credential = true
allowed_services = [{ scheme = "http", host = "127.0.0.1", port = 0,
path_prefix = "/api" }]
[credentials]
env = "CREDPROOF_TEST_CREDENTIAL"
[entry]
module = "tool"
callable = "run"
[[entry.scenarios]]
name = "allowed_file_redirect"
expected_error = "HTTPError"
require_network = true
request = { resource = "demo", path = "__CREDPROOF_ALLOWED_FILE__", url
= "__CREDPROOF_REDIRECT_URL__" }
```

其中 port = 0 表示每次检查动态分配本地端口；场景请求中的占位符在运行时替换为本次的允许文件、跳转地址等；require_network = true 要求该场景必须实际发出请求，防止提前报错冒充安全。

A.4 交付清单

表 A2 交付内容

内容	说明
作品报告	本文档（PDF）
Windows 交付包	源码、前端、示例项目与已脱敏的展示材料，内置官方 Windows 嵌入式 Python 3.12.10，保留其许可
启动脚本	start-credproof.cmd（查看）、start-recheck.cmd（隔离验收）、start-live.cmd（现场）
示例项目	资料助手、bench-v1 八个评测目标、python-dotenv 适配示例、CI 工作流示例
展示材料	bench-v1 冻结结果、对照表、外部组件接入记录、审阅快照与截图
不包含	模型权重、隔离运行时根文件系统、虚拟环境、任何真实凭据

附录二 参赛作品声明

本参赛团队郑重声明：

1. **原创性。** 本作品“密证 CredProof——AI 生成安全补丁的可信验收与人机协同修复平台”的设计方案、核心代码、实验数据与报告由参赛团队独立完成。作品中引用的公开数据、文献和开源组件均已注明出处。
2. **开源组件与许可。** 作品使用的 bubblewrap、pytest、Ollama、Qwen 系列模型、python-dotenv、Gitleaks、Bandit、Semgrep 等第三方组件归各自权利人所有，均按其开源许可使用；交付包未包含模型权重，内置的 Python 运行时保留官方许可。python-dotenv 实验中的缺陷为人工注入，不是上游漏洞。
3. **数据真实性。** 报告中的全部实验数据来自真实运行记录，失败与不利结果均如实保留；标注为“AI 候选”的代码来自本地模型的真实输出，未经修饰；所有凭据均为随机生成的合成值，所有服务均为隔离环境内的本地模拟服务，未对任何真实系统进行测试。
4. **合规与防御性。** 本作品为防御性安全工具，仅用于对已授权项目进行验收，不包含攻击代码。参赛团队遵守大赛规则及国家相关法律法规，如有违反，愿承担相应责任。

参赛团队（签字）：

日期：　　年　　月　　日